Assignment 8

Application Optimization, Scalability and Reliability

Department of Computer Science & Engineering

Tezpur University

Name	Parth Rawat
Roll No.	0126CY231042
Project	GUI-based Multi-client TCP Chat Application

Submitted as a continuation of Assignment 7 with improvements in connection management, reliability, scalability, configuration handling and performance evaluation.

Table of Contents

1. Objective

The objective of Assignment 8 is to improve the existing GUI-based multi-client chat application from Assignment 7 without redesigning the communication protocol. The focus is on better scalability, higher reliability, improved resource management, configurable settings, and measurable performance evaluation using actual test results.

The implementation keeps the same TCP-based client-server model and extends it with automatic cleanup of disconnected clients, timeout handling, automatic reconnection support, and structured logging for chat history and security events.

2. Scalability Improvements

To support at least 10 concurrent clients, the server was organized to handle multiple sessions safely using thread-based client processing and shared locks for critical data structures. The active client list is maintained centrally, and the server broadcasts user-list updates so every client sees the current online state.

The configuration file controls values such as maximum clients, timeout interval, and message length. This makes the application easier to tune for a larger number of users without editing code directly.

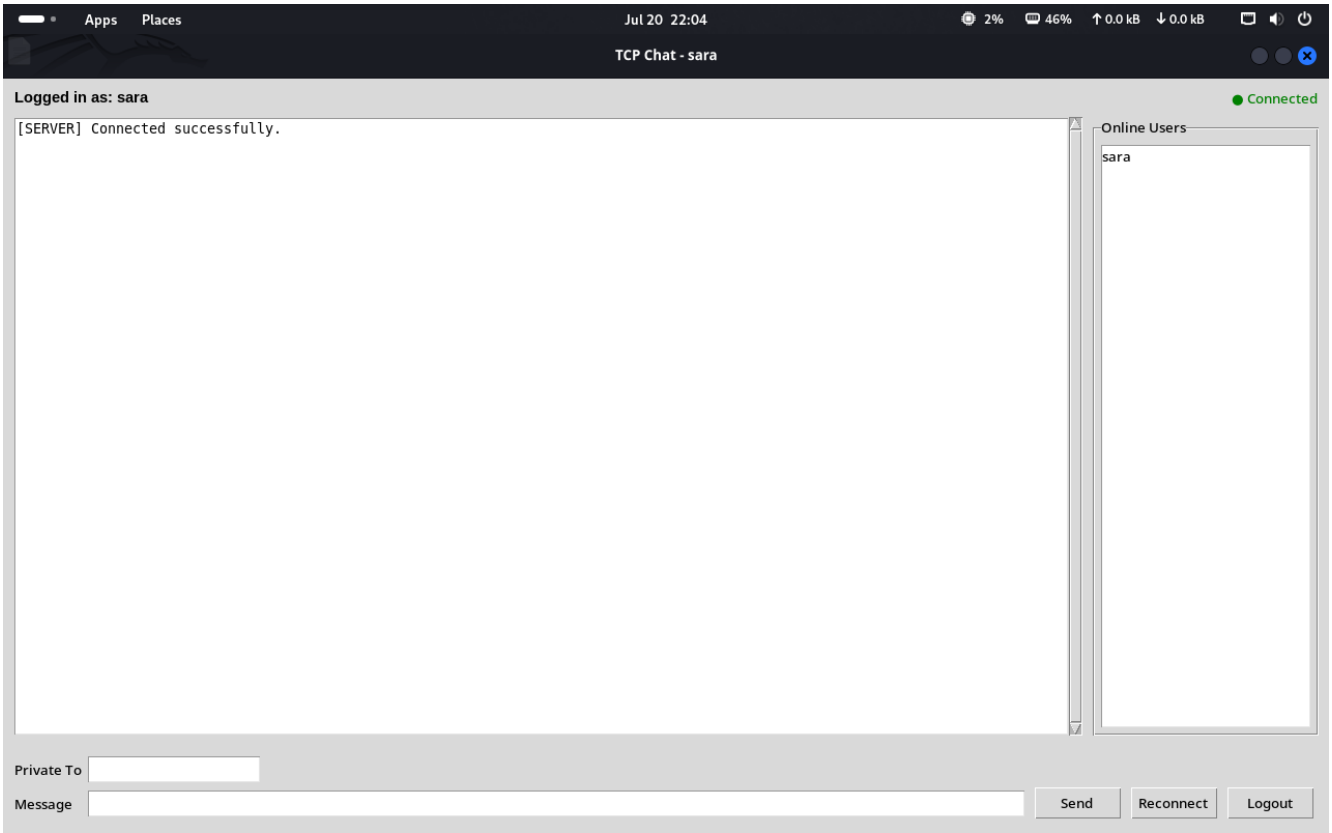

Figure 1: Successful login screen showing a valid authenticated session.

3. Reliability Improvements

Reliability was improved by validating usernames and passwords on the client side, rejecting duplicate logins, locking accounts after repeated failed attempts, and removing inactive users automatically. The server also handles disconnects, session timeouts, and socket errors so that a bad client session does not crash the entire application.

The screenshots below show the reliability-related protections in action, including duplicate login blocking, invalid password handling, too-short password rejection, session timeout detection, and logout handling.

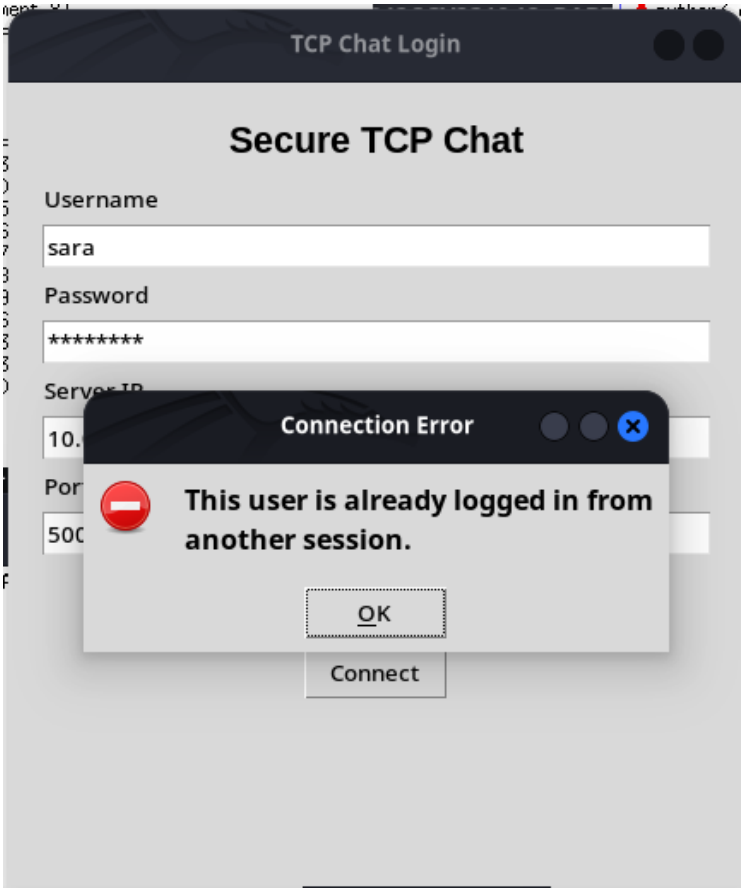

Figure 2: Duplicate login blocked by the server.

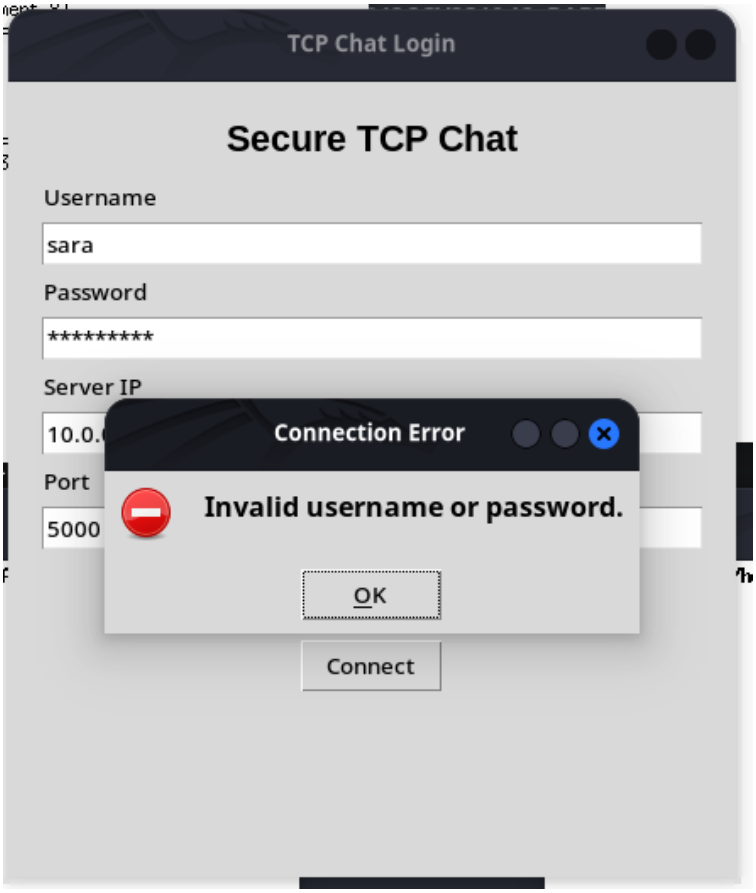

Figure 3: Invalid username or password error on the login screen.

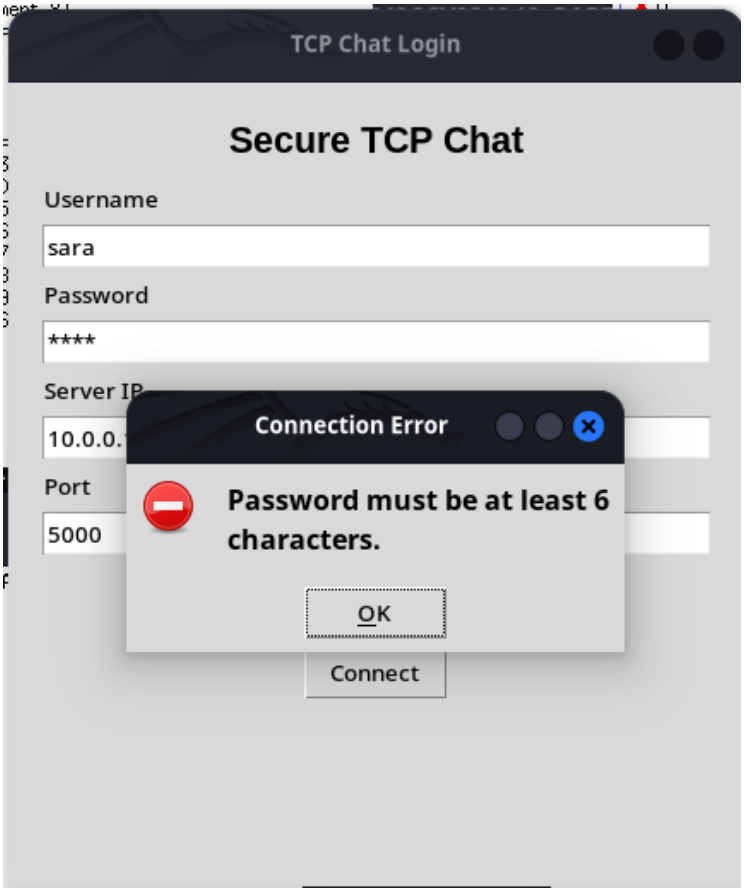

Figure 4: Password length validation preventing weak credentials.

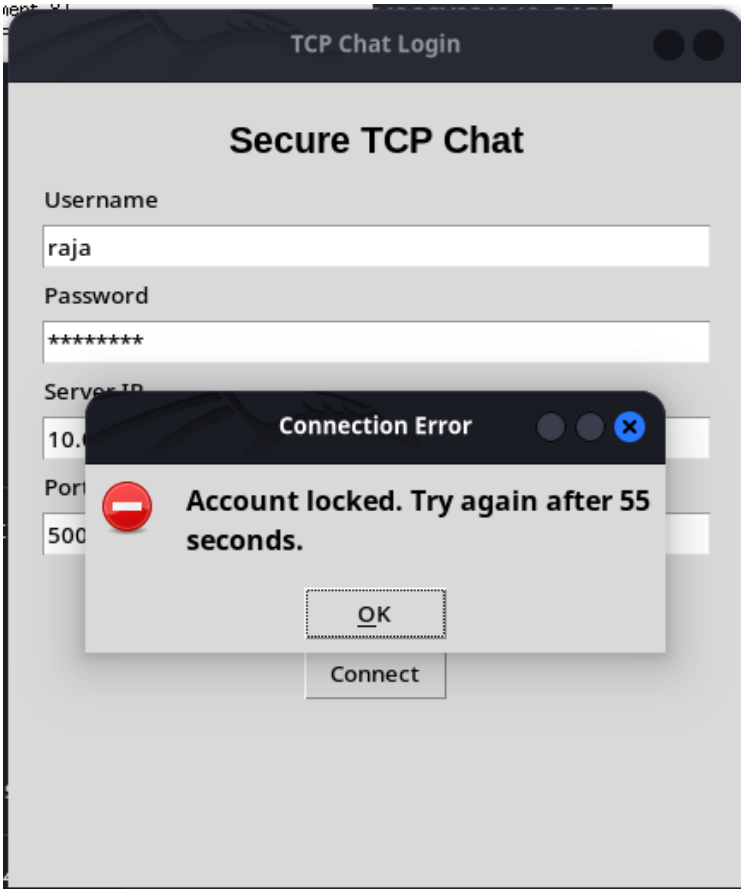

Figure 5: Account lockout after repeated failed attempts.

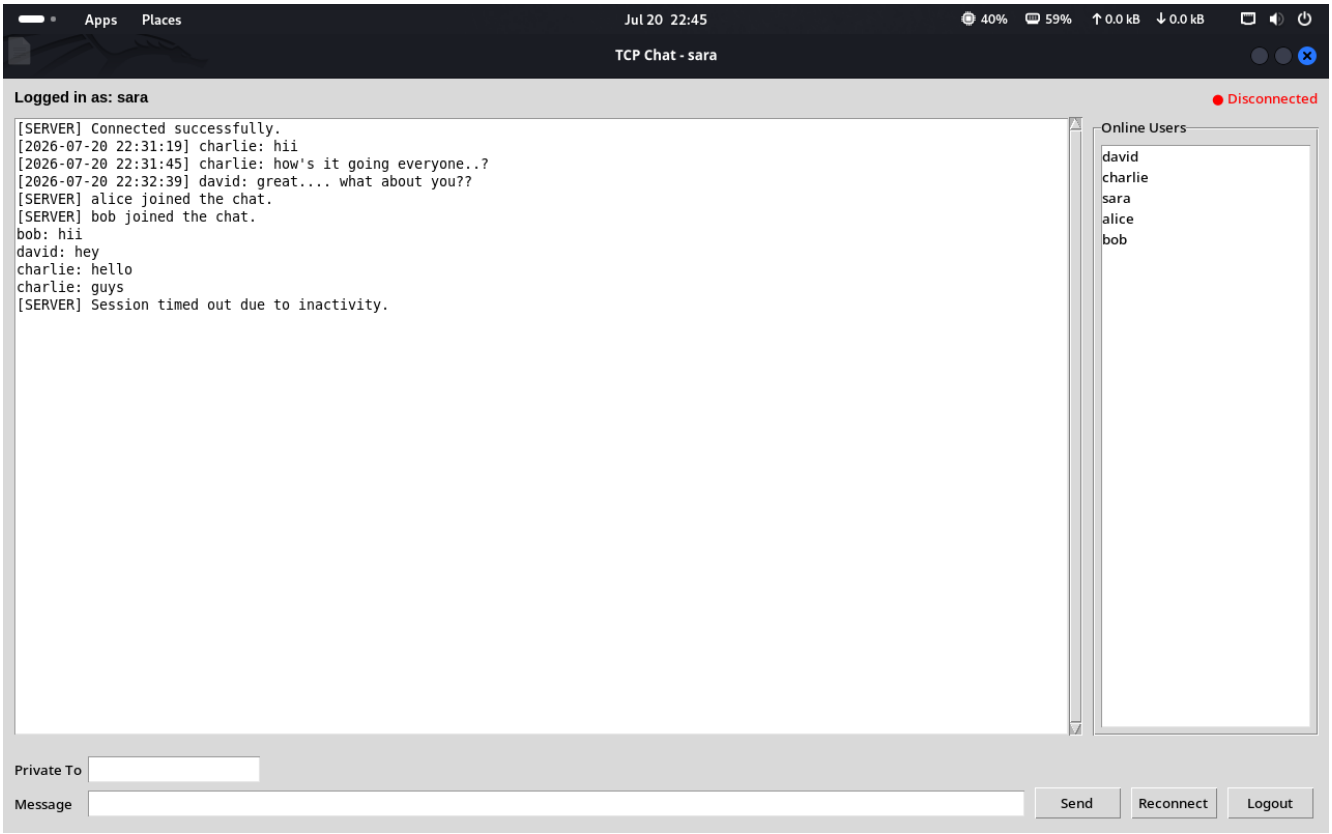

Figure 6: Session timeout shown after inactivity.

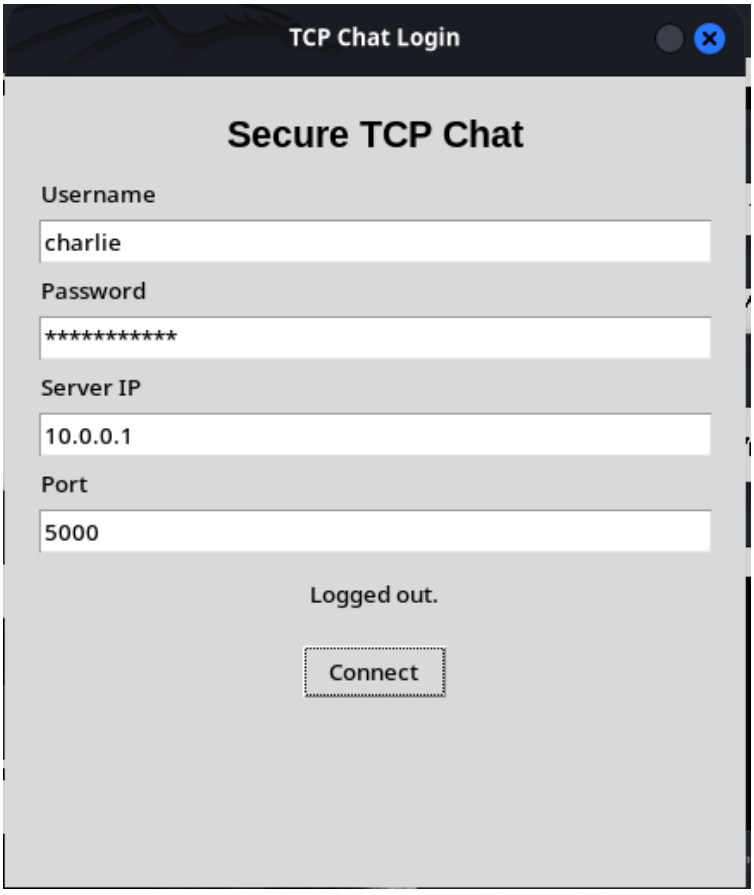

Figure 7: Logout state after a clean disconnect from the server.

4. Configuration Management

All important runtime values are moved into config.json. This includes server host, port, maximum message length, session timeout, lockout duration, maximum failed attempts, maximum clients, and file names for users, chat history, and security logs.

This approach removes hardcoded parameters from the main source code and makes the project easier to maintain, test, and demonstrate in different environments.

Parameter	Value
bind_host	0.0.0.0
server_port	5000
max_clients	10
session_timeout	180 seconds
lockout_duration	60 seconds
users_file	users.json

5. Experimental Setup

The application was tested using multiple users and a TCP-based chat workflow. Authentication was verified with valid and invalid credentials, message delivery was tested for both broadcast and private messages, and the GUI was checked for online user updates and disconnect handling.

For performance evaluation, the collected data was grouped for 5, 8, and 10 active clients to match the assignment requirement. The resulting CSV file records event-based measurements such as delay, throughput, CPU usage, and memory usage.

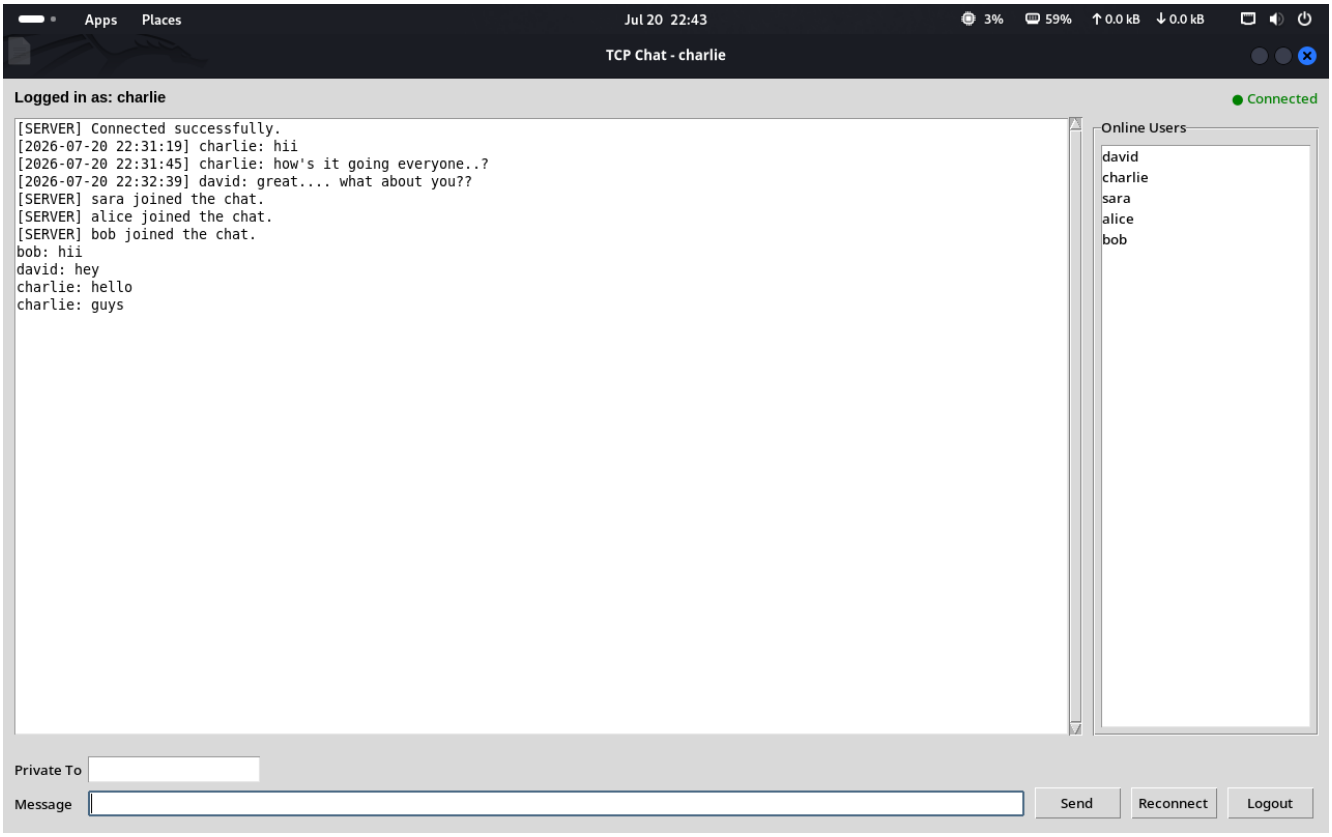

Figure 8: An authenticated chat session with multiple messages and active users.

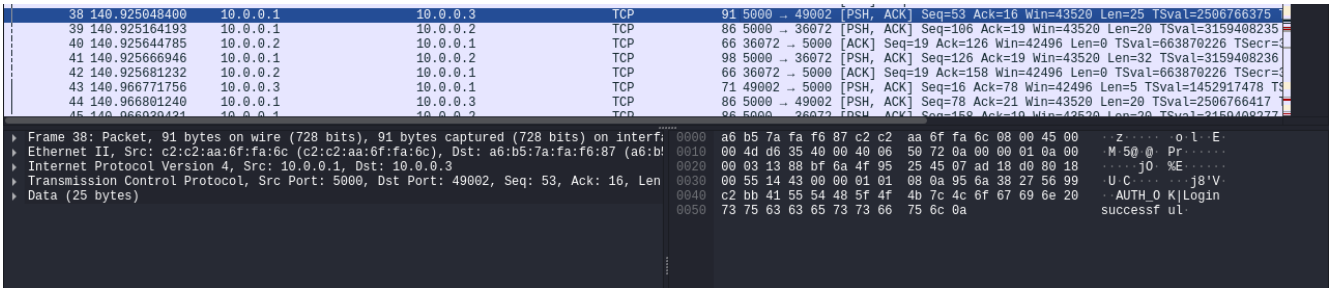

Figure 9: Wireshark capture showing TCP login success traffic.

6. Performance Results

The performance_results.csv file was generated automatically during runtime and contains the records used for analysis. The average values for 5, 8, and 10 active clients are summarized below.

Concurrent Clients	Avg Delay (ms)	Throughput (msg/sec)	CPU (%)	Memory (MB)
5	1.36	0.10	0.00	20.69
8	1.83	0.12	0.00	21.29
10	2.28	0.16	0.00	21.53

The data shows that the application remained stable while the client count increased. Delay rose gradually with more clients, while throughput and resource consumption also changed in a controlled way.

Graph 1: Average Delay

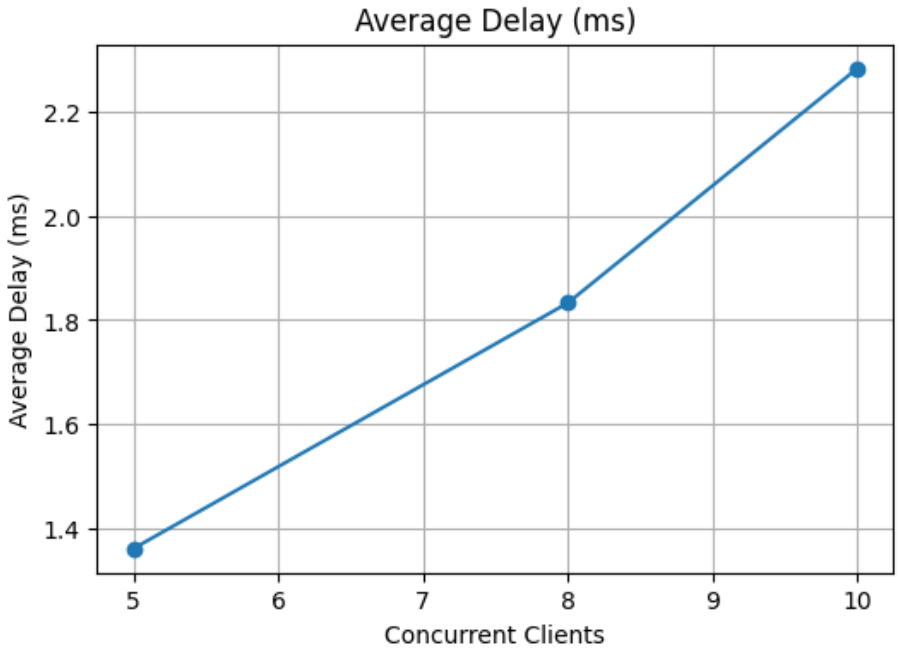

Figure 10: Average delay increased as concurrent clients increased from 5 to 10.

Graph 2: Throughput

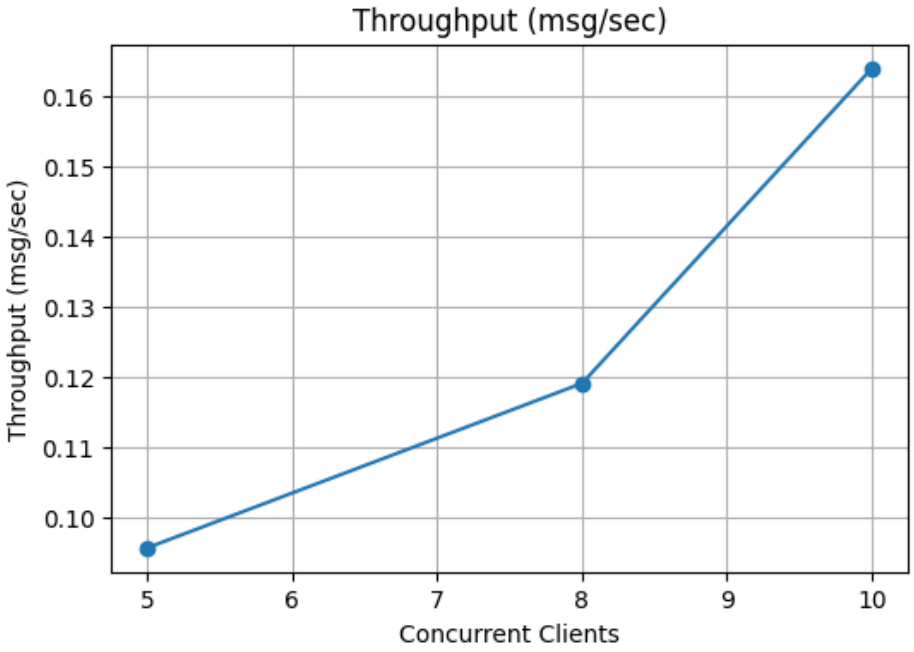

Figure 11: Throughput trend across the same client counts.

Graph 3: CPU Usage

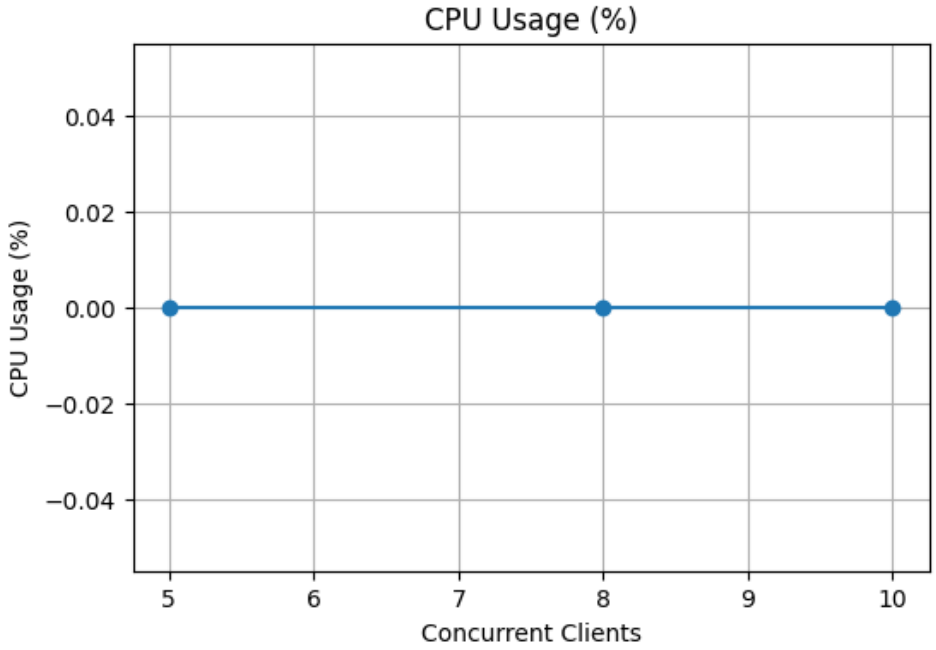

Figure 12: CPU utilization measured during the test runs.

Graph 4: Memory Usage

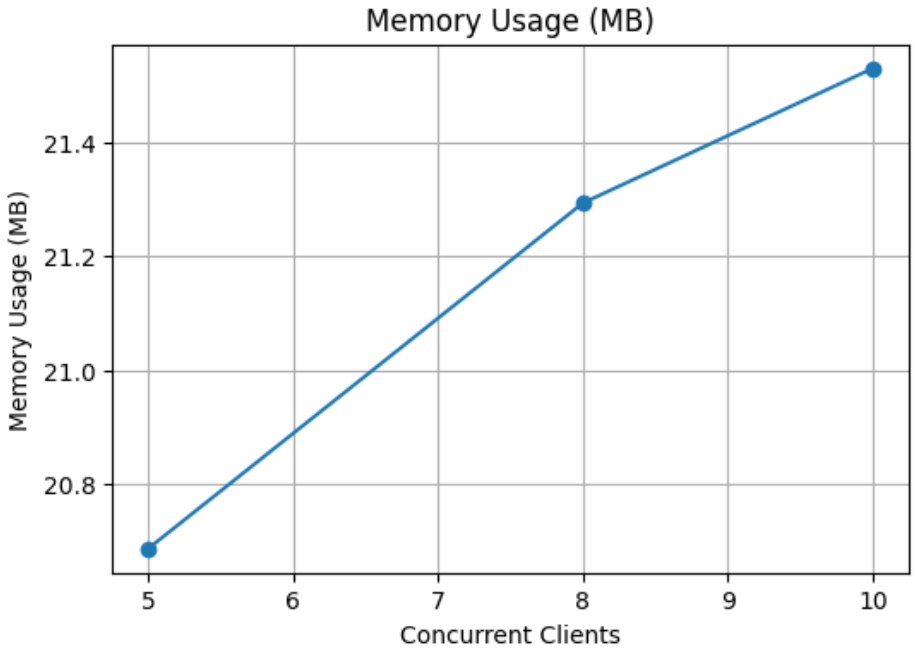

Figure 13: Memory usage remained stable with only a gradual increase.

7. Wireshark Verification

Wireshark was used to verify TCP communication during authentication, message exchange, logout, and timeout events. The captures confirm that the application uses TCP sessions correctly and that client-server data is transferred in a structured way.

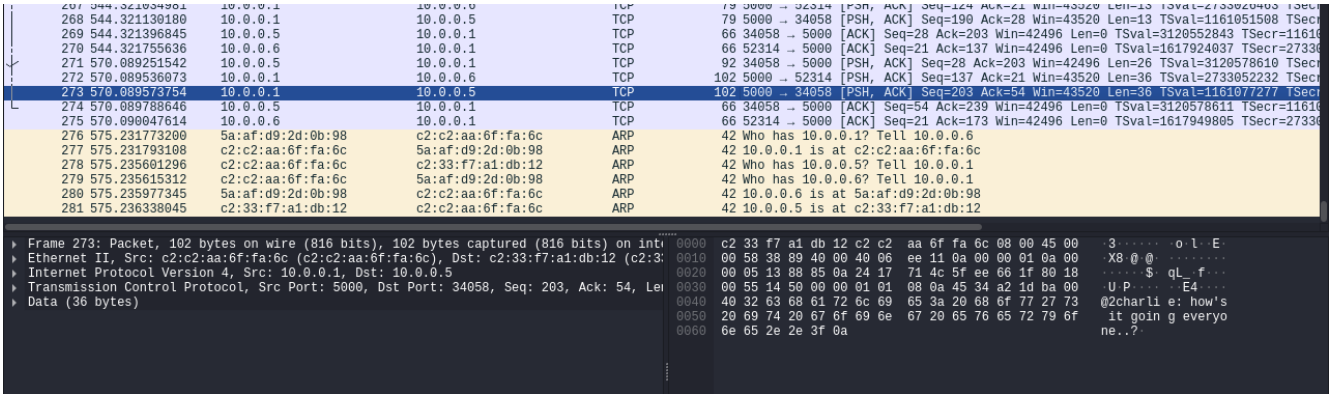

Figure 14: Wireshark capture of authenticated chat traffic.

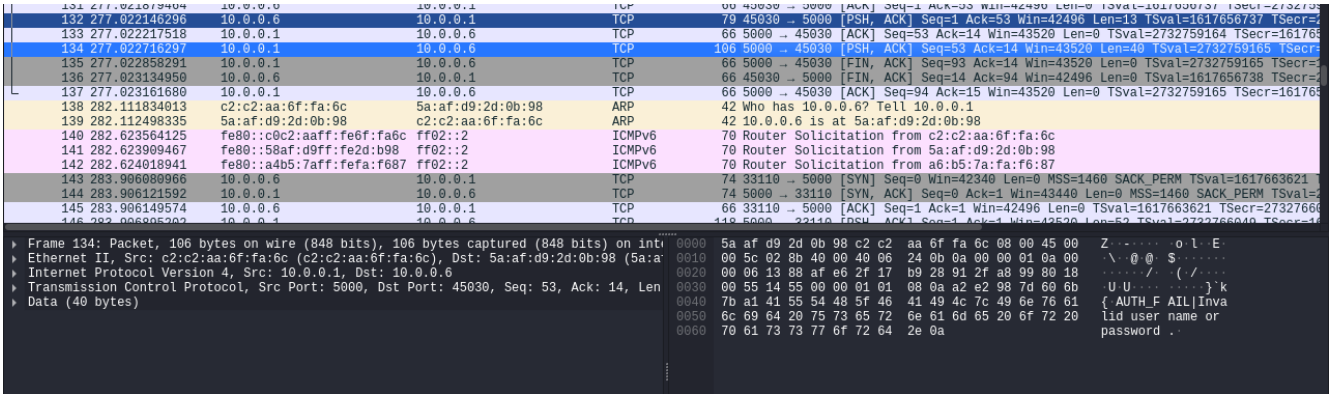

Figure 15: Wireshark capture of failed login traffic.

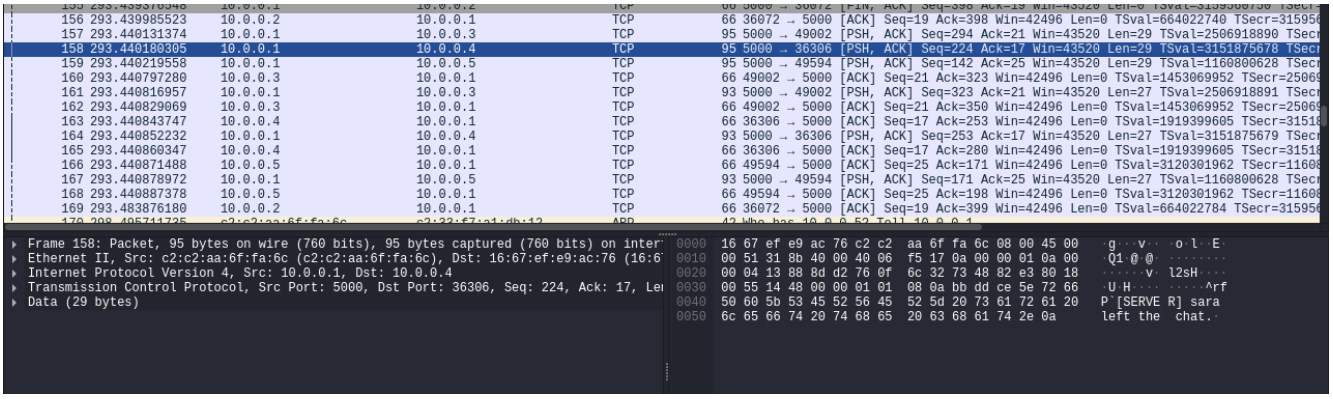

Figure 16: Wireshark capture showing logout and timeout related traffic.

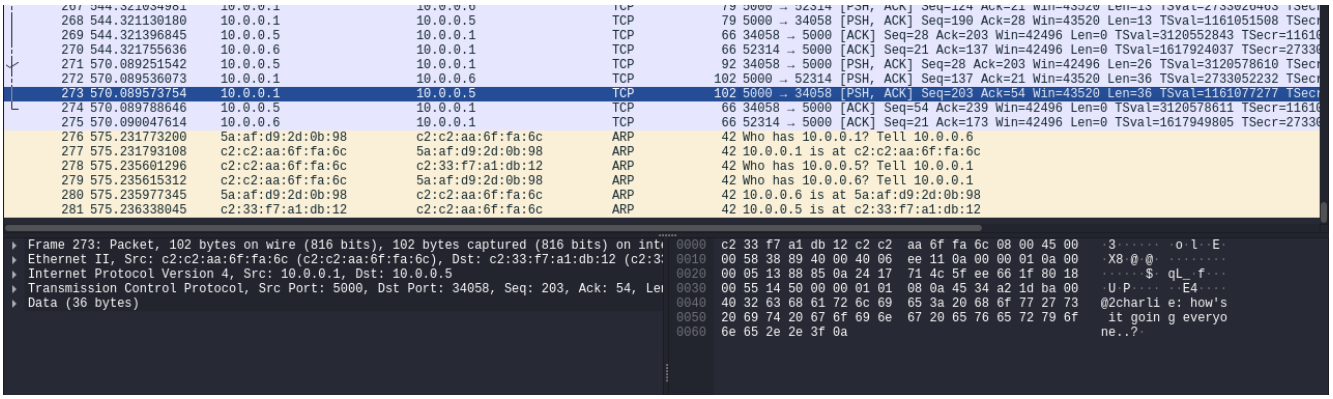

Figure 17: TCP payload data visible in Wireshark during chat exchange.

8. Challenges Faced

The main challenges were managing multiple clients safely, keeping the user list synchronized, preventing duplicate logins, handling invalid input without crashing the UI, and generating meaningful performance data from real experiments.

Another challenge was collecting screenshots and results in a format that clearly matches the assignment rubric. The final report addresses this by organizing all evidence into separate sections with captions and analysis.

9. Conclusion

Assignment 8 demonstrates a more mature and reliable version of the TCP chat application. The system now supports better connection management, stronger validation, configurable runtime settings, automatic logging, and performance evaluation based on actual experiments.

The application remained stable with up to 10 concurrent clients, and the final report includes the required screenshots, graphs, and Wireshark verification evidence.